SYSTEM SPECIFICATION & TECHNICAL ARCHITECTURE

Autonomous Cloud Resource Scheduling & FinOps Governance System

An enterprise automated FinOps platform featuring continuous policy audits, off-hours shutdown enforcement, multi-channel push siren alerts, cryptographic HMAC-SHA256 1-click grace extensions, and dual-engine data persistence.

FRONTEND STACK

HTML5 • CSS3 Custom Tokens • Vanilla JS ES6+ • Bootstrap 5 • Jinja2

BACKEND STACK

Python 3.13 • Flask 3.1+ • APScheduler • Boto3 AWS SDK • Cryptography

DATABASE DUAL-ENGINE

SQLAlchemy 2.0+ • MySQL 8.0 Enterprise • SQLite 3 WAL Fallback

MULTI-CHANNEL ALERTS & NETWORKING

ntfy Siren Push • Twilio SMS/WhatsApp • SMTP Email • Cloudflare Tunnel

Document Contents: Complete Architecture Overview, Technology Stack (Frontend, Backend, Database), Data Model & Schema DDL, Production Deployment (Render, Local Background, Docker, VPS), Governance Policies, Cryptographic Action Security, REST APIs, and Automated Test Protocols.

1. System Overview & Architecture

SECTION 1 • ARCHITECTURE

1.1 Executive Summary & FinOps Challenge

In modern multi-cloud infrastructure (AWS, Azure, GCP), development, testing, and staging compute instances frequently run **24 hours a day, 7 days a week**, even though teams only operate during business hours (8 to 9 hours/day). Over **65% to 70%** of total compute spend is incurred during off-hours, weekends, and holidays. This system solves this challenge by autonomously monitoring compute inventory, validating running instances against allowable schedules, broadcasting urgent multi-channel alerts, providing 1-click mobile cryptographic extensions, and executing safe automated fallback shutdowns.

1.2 End-to-End System Workflow

1. Discovery
AWS Boto3 & Mock Adapter discovers instances into DB

➔

2. Policy Audit
APScheduler 60s background cycle detects off-hour breaches

➔

3. Multi-Channel Alert
Dispatches ntfy siren push, Twilio SMS/WA, and SMTP email

➔

4. 1-Click Action
HMAC token: Stop instance or Extend lease by 2 hours

➔

5. Auto-Shutdown
If unacknowledged for 30m, safe fallback shutdown executes

1.3 Key Architectural Modules

🛡️ Policy & Scheduler Engine (APScheduler)

Continuously evaluates schedules using UTC timestamps. Identifies un-tagged, idle, or after-hours instances. Manages automated 30-minute fallback shutdown countdowns to eliminate cost leakage.

🔐 Cryptographic Action Gateway (HMAC-SHA256)

Generates time-limited (15-min TTL) cryptographically signed URLs for Stop and Extend actions. Eliminates the need for engineers to log into the AWS Console; supports single-click execution from lockscreen notifications.

📢 Real-Time Multi-Channel Dispatcher

Dispatches alerts simultaneously via **ntfy Mobile Push** (with audible emergency buzzer sirens), **Twilio SMS & WhatsApp**, and **Gmail SMTP** with styled HTML action buttons.

🌐 Cloudflare Edge Ingress Tunnel

Integrates `cloudflared.exe` to expose the local host on a secure HTTPS domain (`https://*.trycloudflare.com`), enabling remote smartphone access over 4G/5G mobile networks.

2. Complete Technology Stack

2.1 Frontend Technologies (Client-Side)

Technology	Version / Type	Role & Functional Implementation
HTML5	Semantic Web Standard	Structures all operational dashboards, KPI cards, resource tables, schedule dialogs, and mobile confirmation views.
CSS3 & Custom Tokens	Custom Design System	CSS variables, glassmorphism, responsive flex/grid layouts, card border accents, and micro-animations in `style.css`.
JavaScript (ES6+)	Vanilla Asynchronous JS	Dynamic client-side controller in `dashboard.js`; manages 20s telemetry auto-refresh, fetch API calls, and DOM updates.
Bootstrap 5	v5.3+ (Dark/Light Console)	Executive UI components, responsive layout grid, form controls, tables, and modal dialogs.
Jinja2	Flask Templating Engine	Server-side HTML template inheritance (`base.html`), conditional RBAC rendering, and session token injection.
FontAwesome 6	v6.5+ (CDN)	Vector iconography for cloud providers (AWS, Azure, GCP), status badges, and action triggers.
Google Fonts	Web Fonts Stack	**Plus Jakarta Sans** (Headings/UI), **Inter** (Body text), and **JetBrains Mono** (Telemetry/IPs).

2.2 Backend Technologies (Server-Side)

Component	Technology & Version	Role & Functional Implementation
Language	Python 3.13 / 3.11+	High-performance, type-hinted core server runtime executing web requests and asynchronous scheduler threads.
Web Framework	Flask 3.1+	Application Factory pattern with Blueprints: `auth_bp`, `dashboard_bp`, `api_bp`, and `webhook_bp`.
Background Scheduler	APScheduler 3.10+	In-process multi-threaded `BackgroundScheduler` running 60-second periodic policy audit loops and auto-shutdown timers.
Cloud SDK	AWS Boto3 1.34+	Interacts with live AWS EC2 APIs (`describe_instances`, `start_instances`, `stop_instances`).
Simulation Engine	Mock Cloud Adapter	Pre-seeded with realistic EC2 instances for high-fidelity offline demonstrations without incurring cloud billing.
Security & Hashing	Bcrypt 4.1+ & HMAC-SHA256	Bcrypt for password hashing (12 rounds) and HMAC-SHA256 for signing tamper-proof single-click action tokens.
Notification Gateways	ntfy.sh / Twilio / SMTP	Multi-channel alert dispatching across mobile push sirens (ntfy), SMS/WhatsApp (Twilio), and HTML email (Gmail TLS).
Edge Tunnel	Cloudflare Tunnel	Embedded `cloudflared.exe` mapping port 5000 to public HTTPS for mobile webhook callbacks over cell networks.
Production Server	Gunicorn 21.2+	Production WSGI HTTP server supporting multi-worker execution on Linux and cloud platforms.

3. Database & Data Model Architecture

SECTION 3 • DATABASE

3.1 Dual-Engine Database Architecture

The system employs a **Dual-Engine Architecture** managed through **SQLAlchemy 2.0+ ORM** with thread-safe `scoped_session` management. It connects to MySQL in production, and automatically falls back to SQLite if MySQL is offline:

🐬 Primary: MySQL 8.0+ Enterprise

- Driver: `pymysql 1.1+`
- Connection pool: `pool_pre_ping=True`, `pool_recycle=3600`
- Complete DDL script provided in `schema.sql` with foreign key cascades and indexes.

🗂️ Automatic Fallback: SQLite 3

- Database file: `cloud_scheduler.db` (zero manual configuration)
- Optimized with high-concurrency **WAL (Write-Ahead Logging)** mode
- Pragma: `synchronous=NORMAL`, `cache_size=10000`.

3.2 Database Entity-Relationship (ER) Schema

Table	Key Columns	Functional Role & Relationships
users	`id, name, email, password_hash, phone_number, role, created_at`	RBAC user accounts. Roles: `Admin`, `Manager`, `Viewer`. Passwords hashed with Bcrypt.
cloud_resources	`id, instance_id, instance_name, provider, region, state, public_ip, private_ip, launch_time, lease_expires_at, alert_sent_at`	Discovered compute instances. Tracks operational state (running/stopped), continuous uptime hours, and active lease expirations.
schedules	`id, resource_id, start_time, end_time, max_runtime_hours, days_of_week, is_active`	Permitted execution policies per resource. Supports both daily recurring windows (HH:MM) and calendar timestamps (DD-MM-YYYY HH:MM).
alert_logs	`id, resource_id, channel, recipient, status, token_hash, details, sent_at`	Audit log of outbound alerts dispatched across ntfy, Twilio SMS, WhatsApp, and SMTP Email with delivery statuses.
audit_trails	`id, resource_id, triggered_by, action_taken, details, timestamp`	Immutable ledger of state changes: START, STOP, EXTEND_LEASE, SCAN, POLICY_SHUTDOWN, and operator actions.
cloud_accounts	`id, provider, account_name, account_identifier, region, credentials_json, status, is_active`	Encrypted multi-cloud credentials for connecting live AWS, Azure, and Google Cloud Platform environments.

3.3 FinOps Cost Calculation Formulas

💵 Active Cost Burn Calculation

```
Active Hourly Burn ($) = Total Running Instances × Hourly Rate ($0.096)
Active Hourly Burn (₹) = Active Hourly Burn ($) × USD_TO_INR_RATE (83.50)
```

🛡️ Protected Capital / Total Savings

```
Cumulative Savings ($) = Stopped Instances × Off-Hours Saved × Hourly Rate
Cumulative Savings (₹) = Cumulative Savings ($) × 83.50
```

4. Comprehensive Deployment Guide

4.1 Method 1: Cloud Deployment on Render (Free Hosting Pre-Configured)

The repository includes a ready-to-use `render.yaml` blueprint and `Procfile` for one-click deployment:

1. **Push to GitHub:** Initialize git and push the repository to GitHub:

```
git init && git add . && git commit -m "Deploy Cloud Scheduler"
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

2. **Connect to Render:** Log in to render.com, click **New +** > **Blueprint**, and select the repository.
3. **Configure Environment Variables:** Set the following required variables in the Render Dashboard:
   - `WEBHOOK_BASE_URL`: `https://<your-app-name>.onrender.com` (enables remote mobile action clicks)
   - `USE_SQLITE_FALLBACK`: `True`
   - `USE_MOCK_AWS`: `True` (or `False` if passing live AWS keys)
   - `NTFY_TOPIC`: `vasanth_cloud_alerts`
4. **Deploy:** Render runs `pip install -r requirements.txt` and executes `python run.py`.

4.2 Method 2: Local 24/7 Deployment with Cloudflare Tunnel (Windows)

For running the system continuously on a host machine with zero cloud costs while allowing remote mobile access:

> 🚀 **1-Click Silent Background Host**
>
> Double-click `START_BACKGROUND.bat`. It spawns the server as a background process and launches the Cloudflare Tunnel. The scheduler and phone push alerts continue running 24/7 even after Antigravity IDE is closed.

> 🛑 **Stopping the System**
>
> Double-click `STOP_SYSTEM.bat`. It cleanly terminates any process bound to port 5000 and kills running `cloudflared` tunnel processes.

4.3 Method 3: Containerized Deployment (Docker)

The included `Dockerfile` provides a production container based on `python:3.13-slim`:

```
# 1. Build the Docker container image
docker build -t cloud-resource-scheduler .

# 2. Run container in detached mode with persistent database volume
docker run -d --name cloud-scheduler -p 5000:5000 --env-file .env -v scheduler-db:/app/instance --restart unless-stopped clo
ud-resource-scheduler
```

4.4 Method 4: Production Linux Server (Ubuntu / AWS EC2)

```
# 1. Install system runtime & virtual environment
sudo apt update && sudo apt install -y python3 python3-pip python3-venv git
git clone <repo-url> /opt/cloud-scheduler && cd /opt/cloud-scheduler
python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt

# 2. Run with Gunicorn production server
gunicorn --workers 3 --bind 0.0.0.0:5000 "run:app"
```

5. Security, Webhooks & Automated Testing

SECTION 5 • SECURITY & QUALITY

5.1 Cryptographic Action Tokens (HMAC-SHA256)

Single-click remote governance actions (Stop and Extend) operate without requiring engineers to enter passwords or log into the cloud console:

- **Token Structure:** Base64 URL-safe encoded payload containing `instance_id`, `action`, `nonce`, and `exp` timestamp.
- **Cryptographic Signature:** Signed using HMAC-SHA256 with the server's `SECRET_KEY`.
- **Time-To-Live (TTL):** Strict 15-minute validity window (`ACTION_TOKEN_TTL_SECONDS=900`). Expired tokens are immediately rejected.
- **Replay & Tamper Protection:** Altering any single byte invalidates the signature; tokens are consumed on execution to prevent replay attacks.

5.2 Core REST API Endpoints

Endpoint	Method	Auth Level	Description
`/quick-login`	GET	Public	1-Click instant administrator authentication for fast access during demonstrations.
`/api/resources`	GET / POST	Session (Admin/Manager)	Retrieves discovered cloud compute instances or initiates manual cloud inventory sync.
`/api/resources/<id>/action`	POST	Session (Admin/Manager)	Triggers manual START, STOP, or EXTEND action on an EC2 instance.
`/api/schedules`	GET / POST	Session (Admin)	Reads or configures operating hours, runtime caps, and active days per resource.
`/api/audit-policies`	POST	Session (Admin)	Manually triggers an immediate policy audit cycle outside the 60s background loop.
`/webhook/action`	GET / POST	HMAC Token	Validates cryptographic token and executes remote 1-click Stop or 2-Hour Lease Extension.

5.3 Automated Pytest Test Suite

The project includes **16 passing automated tests** covering all critical policy and security paths:

```
# Run the full automated test suite with verbose output
python -m pytest -v tests
```

test_tokens.py
- HMAC signature generation
- Tamper detection rejection
- 15-minute TTL expiration
- Replay prevention verification

test_scheduler.py
- EC2 discovery & state transition
- Schedule window compliance
- Policy breach alert dispatch
- 30m fallback auto-shutdown

test_webhook.py
- GET 1-click STOP execution
- POST 2-hour lease extension
- Mobile HTML response confirmation
- Invalid token error rejection

5.4 Default System Credentials

Primary Admin Account
- **Email:** `vasanthkumar20212021@gmail.com`
- **Passkey:** `password123` (or `Admin@123456`)
- **1-Click Instant Login:** Navigate directly to `/quick-login`

Emergency Alert Dispatch Configuration
- **ntfy Push Channel:** `https://ntfy.sh/vasanth_cloud_alerts`
- **Alert Phone / WhatsApp:** `+919025903459`
- **Recipient Email:** `vasanthkumar20262026@gmail.com`